Information Technology Course
Module Cloud Computing by
Damir Dobric

ML25/26-11 Implement Reminder Agent

KavyaSree, Thabjul -1633324 (thabjul.sree@stud.fra-uas.de)
Bavana Jadala-1629828 (bavana.jadala@stud.fra-uas.de)
Aishwarya Rajendra Sawant-1629688 (aishwarya.sawant@stud.fra-uas.de)

Abstract — ReminderAgent is a conversational AI system for saving and retrieving personal reminders restaurants, books, travel plans through free-form natural language, built on the Microsoft Agent Framework with hybrid retrieval combining metadata filtering and 1,536-dimensional OpenAI cosine similarity search. This cloud computing project migrates the console-based implementation to Microsoft Azure, preserving the original console application unchanged and extending the system with a new Model Context Protocol (MCP) Server deployed as a Docker container on Azure App Service over Streamable HTTP. Local flat-file storage is replaced with Azure SQL Database (VECTOR (1536) embedding column), Azure Blob Storage, Azure Queue Storage, and Azure Table Storage. Nine MCP tools are exposed through the shared ReminderTools library and verified against GitHub Copilot Chat and Claude Desktop, and the MCP Server is deployed and confirmed live on Azure App Service as the single, network-accessible entry point to the system.

***Keywords*— Azure SQL, Azure Blob Storage, Azure Queue Storage, Azure Table Storage, Model Context Protocol, OpenAI, Semantic Search, Vector Embeddings, .NET 10.0, AI-Assisted Development, C#**

I. INTRODUCTION

A. Problem Statement

Personal information management tools fail at both ends of the task. At capture time they demand categories, tags, and structured fields. At retrieval time they require the user to recall the exact words used when saving a requirement people routinely cannot meet [19] . Reminder Agent was built in the Software Engineering project [1] . using the Microsoft Agent Framework to solve this. It [15] accepted free-form natural language for saving and retrieving personal assets such as restaurants, books, hotels, and travel destinations stored them in CSV and JSON files and used hybrid retrieval: metadata filtering for concrete queries and cosine similarity over 1,536-dimensional OpenAI embeddings for vague ones [2] . The principal Software Engineering limitations were linear cosine scan scalability, single-user storage isolation, and no ability to expose tools to remote AI clients without cloning and running the project locally.

B. Goals and Scope

This project has five objectives. The first is to replace the flat-file CSV and JSON storage of the SE implementation with Azure SQL Database, storing both relational asset data and 1,536-dimensional vector embeddings in a native VECTOR (1536) column so that embeddings persisted alongside the data they describe rather than maintained as a separate sidecar file. The second is to replace the local Photos/ folder with Azure Blob Storage, providing a private container with permanent Blob URLs and time-limited SAS token generation so that photos are securely accessible from anywhere without exposing storage credentials. The third is to add asynchronous bulk import of CSV, JSON, and ZIP files via Azure Queue Storage with a background worker and a three-retry poison-queue mechanism, so that importing large batches of reminders does not block the user and failed messages are captured for inspection rather than lost or retried indefinitely. The fourth is to migrate the system to a single, network-accessible production interface: the original SE-project console application is retained purely as an internal development and testing tool it lets the team exercise ReminderTools business logic directly, independent of MCP protocol concerns, during implementation but it is not packaged, containerized, or exposed in production. The new ReminderAgent.McpServer project is the only interface end users and AI clients interact with, so there is exactly one production entry point and no duplicated business logic across parallel interfaces.

The fifth is to deploy the MCP Server as a Docker container on Azure App Service with a real Model Context Protocol Server over Streamable HTTP, so that any MCP-compatible AI client GitHub Copilot Chat, Claude Desktop, Cursor, or VS Code can connect and use all tools over the network without cloning the repository or running the project locally.

C. Technology Choices

Microsoft Azure was selected as the cloud platform for three reasons. First, Azure SQL Database natively supports the VECTOR (1536) column type in its preview release, enabling first-class storage of 1,536-dimensional embedding vectors directly alongside relational asset data without requiring a separate vector database or an external indexing service. Second, all four required storage primitive's relational database, blob storage, queue storage, and table storage are available within a single Azure resource group, simplifying access control, billing, and connection string management. Third, Azure App Service provides first-class Docker container hosting on Linux, matching the .NET 10.0 runtime used throughout the project.

The Model Context Protocol was chosen over a custom REST API for AI client integration because MCP clients discover and invoke available tools autonomously at runtime from [Description] attributes decorated on each tool method [9] .

This means no bespoke client integration is required any MCP-compatible client connects to the server URL and immediately knows which tools exist, what parameters they accept, and what they do, without any additional configuration or documentation exchange.

OpenAI was selected for both language understanding and embedding generation. GPT-4o-mini handles natural language routing in the console REPL entry point [16] and text-embedding-3-small generates the 1,536-dimensional semantic vectors stored in Azure SQL. Both models are accessed through the Microsoft.Extensions.AI abstraction layer, which decouples the application from any specific model provider and allows future substitution for example, replacing OpenAI with Azure OpenAI Service or another provider without changing a single line of application code.

This implementation is a single MCP-based production entry point backed by one shared tool library, with the SE-project console kept only as an internal development harness was chosen over an approach that would have shipped both interfaces to production. A dual-interface production deployment would have satisfied the deployment requirement, but it would have duplicated the attack surface and operational footprint without adding capability, since every MCP-compatible client already reaches the console's functionality through the server. Keeping the console as a local, non-shipped development tool still lets the team validate ReminderTools business logic directly during development, independent of MCP protocol concerns, while ensuring production traffic has exactly one path into the system: the MCP Server.

Azure Queue Storage was retained specifically for the BulkUploadAssets tool because MCP tool calls are inherently synchronous request-response exchanges a client sends a call and waits for a JSON-RPC response. Processing a large CSV or ZIP file synchronously within that call would block the client for an unpredictable duration and risk a request timeout on either the client or the App Service side. Enqueuing the job and returning a Job ID immediately keeps every MCP tool call fast and predictable, while the actual file processing happens asynchronously in a background worker regardless of which entry point triggered it.

II. LITERATURE REVIEW

A. Conversational AI and Intent Routing

The use of large language models as intent routers over typed tool functions was formalised by the ReAct framework [3] , where the model reasons about what action to take, invokes a tool, observes the result, and reasons again rather than generating a response in one shot. This contrasts with earlier task-oriented dialogue systems, which required hand-crafted state machines and slot-filling grammars to interpret user requests. The Microsoft Agent Framework brings this pattern into the .NET ecosystem through IChatClient, AIFunctionFactory, and UseFunctionInvocation() middleware [4] In ReminderAgent, each tool method is registered with a [Description] attribute that the LLM reads at routing time to determine which method to invoke and what arguments to extract, so the routing contract lives directly alongside the implementation.

B. Retrieval-Augmented Generation

Lewis et al. [2] introduced Retrieval-Augmented Generation as a mechanism for grounding LLM output in retrieved factual context, reducing hallucination on knowledge-intensive tasks. The core insight is that a language model should retrieve relevant context at query time rather than relying solely on parametric memory. In ReminderAgent, retrieval and generation are kept as separate steps: a tool fetches and filters assets from Azure SQL, and a second GPT-4o-mini call converts the structured result into natural-language prose for the console entry point. This separation keeps retrieval correctness independently verifiable from generation quality.

C. Vector Embeddings for Semantic Search

Devlin et al [5] showed that context-aware token representations significantly outperform keyword matching for semantic similarity, a fundamental departure from sparse representations that treat each word independently and cannot capture synonymy or paraphrase. Reimers and Gurevych [6] extended this to sentence-level embeddings using Siamese network fine-tuning, making it practical to compare full sentences by cosine distance. OpenAI text-embedding-3-small produces 1,536-dimensional vectors optimised for retrieval [7] ReminderAgent constructs a composite embedding string per asset by concatenating Name, Category, UserExperience, UserInput, and Tags before encoding, ensuring the resulting vector captures the full semantic context of the asset rather than any single field in isolation.

D. Adaptive Retrieval Routing

Jeong et al [8] showed that selecting a retrieval strategy based on query complexity outperforms a single fixed approach across all query types of simple factual queries benefit from direct retrieval while complex or ambiguous queries benefit from more elaborate pipelines.

III. METHODS

A. System Design

The solution is organized into five .NET 10.0 projects following a clean architecture pattern in which each layer has a single, well-defined responsibility. The projects reference one another in one direction only: ReminderAgent.Common has no project references and is used by all four other projects; ReminderAgent.Infrastructure references only Common; ReminderAgent.Cloud, ReminderAgent.McpServer, and Tests each reference Infrastructure. No project ever

references back down to a project that depends on it, which keeps the codebase easy to reason about and prevents circular dependencies.

ReminderAgent.Common sits at the base of this reference chain and contains no Azure SDK package references of any kind. An Azure SDK is the official .NET client library Microsoft publishes for talking to a specific cloud service for example, Azure.Storage.Blobs is the SDK for Blob Storage and Microsoft.Data.SqlClient is the SDK for SQL Database. ReminderAgent.Common defines all six service interfaces IDataAccessLayer, IPhotoStorageProvider, IQueueService, ITableStatusService, IEmbeddingService, and ISimilarityService as well as the Asset entity class and shared helpers, using only plain C# with no cloud-specific types. Because every other project depends on Common but Common depends on nothing, these interface contracts act as the single authoritative specification that both implementations and tests must satisfy.

ReminderAgent.Infrastructure is the only project in the solution that references any Azure SDK package Azure.Storage.Blobs, Azure.Storage.Queues, Azure.Data.Tables, and Microsoft.Data.SqlClient. It implements all six interfaces defined in Common using these SDKs, and it contains ReminderTools, the class that implements all nine MCP tool methods. Concentrating every Azure SDK reference in this single project means that if a cloud provider SDK needs to be upgraded or replaced for example, moving from Azure.Storage.Blobs to a newer major version the change is made in one project only. ReminderAgent.Cloud, ReminderAgent.McpServer, and Tests all consume Infrastructure only through the Common interfaces, so they are never aware of which concrete SDK or version is in use underneath.

ReminderAgent.McpServer is the single production host application. It depends on Infrastructure and consumes the ReminderTools class directly. It is an ASP.NET Core application that hosts the MCP protocol layer over Streamable HTTP, exposing all nine tools at the /mcp endpoint, and it also runs the background bulk-upload queue worker in the same process so that jobs enqueued through the MCP Server are always picked up.

The original ReminderAgent.Cloud console application from the SE project is retained only as an internal development and testing harness, used by the team to exercise ReminderTools methods directly during implementation. It is excluded from the production build and container image via. dockerignore and is never deployed or exposed to end users or MCP clients.

Tests is the fifth project, depending on Infrastructure so it can instantiate real Azure service implementations and the full DI container. Its ClassInitialize methods build the identical ServiceCollection used by Program.cs, which means the test suite exercises the same wiring, the same service lifetimes, and the same configuration loading that production code uses.

B. Architecture — MCP Server on Azure App Service Points

The deployed system runs ReminderAgent.McpServer as a Docker container on Azure App Service. This is the only entry point exposed to users in production. Figure 1 shows the complete cloud architecture. The user message is received by an Agent a component in the MCP client application such as VS Code GitHub Copilot or Claude Desktop which brings its own LLM model and selects the appropriate tool in-process. The MCP Client then sends a network call over HTTPS to the MCP Server running on Azure App Service. Inside the container, the server receives the already-structured tool name and arguments, looks up the corresponding method in MCP Tools (ReminderTools), and runs it in-process. As labelled in the diagram: every MCP call ends inside the container, and the response goes straight back to the Agent. The server never sees plain language and never makes a routing decision itself.

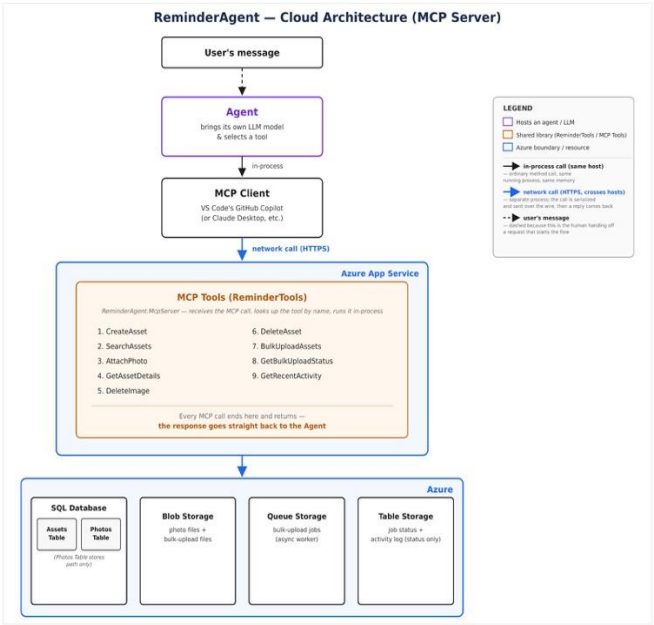


Figure 1 ReminderAgent Cloud Architecture (MCP Server). The user message is received by the Agent (client-side LLM) which selects a tool in-process. The MCP Client sends a network call over HTTPS to the MCP Server deployed on Azure App Service. The server looks up the tool by name and runs it in-process via MCP Tools (ReminderTools). Every call end *inside the container and the response goes straight back to the Agent. Azure services (SQL, Blob, Queue, Table) are shared at the bottom.*

C. MCP Tools (ReminderTools) — The Tool Execution Layer

Inside the Azure App Service container, after the MCP Server receives the incoming tool call, it immediately delegates to MCP Tools (ReminderTools) the orange-bordered box in Figure 1 ReminderAgent Cloud Architecture (MCP Server). The user message is received by the Agent (client-side LLM) which selects a tool in-process. The MCP Client sends a network call over HTTPS to the MCP Server deployed on Azure App Service. The server looks up the tool by name and runs it in-process via MCP Tools (ReminderTools). Every call end inside the container and the response goes straight back to the Agent. Azure services (SQL,

Blob, Queue, Table) are shared at the bottom. labelled as the shared library. ReminderTools is not Azure-specific; it is the single class that implements all nine tool methods. The nine tools listed in the diagram are: CreateAsset, SearchAsset, AttachPhoto, GetAssetDetails, DeleteImage, DeleteAsset, BulkUploadAssets, GetBulkUploadStatus, and GetRecentActivity. As noted in Figure 1 ReminderAgent Cloud Architecture (MCP Server). The user message is received by the Agent (client-side LLM) which selects a tool in-process. The MCP Client sends a network call over HTTPS to the MCP Server deployed on Azure App Service. The server looks up the tool by name and runs it in-process via MCP Tools (ReminderTools). Every call end inside the container and the response goes straight back to the Agent. Azure services (SQL, Blob, Queue, Table) are shared at the bottom. every MCP call ends here, and the response goes straight back to the Agent. ReminderTools then reads from or writes to the four Azure services shown at the bottom of Figure 1 ReminderAgent Cloud Architecture (MCP Server). The user message is received by the Agent (client-side LLM) which selects a tool in-process. The MCP Client sends a network call over HTTPS to the MCP Server deployed on Azure App Service. The server looks up the tool by name and runs it in-process via MCP Tools (ReminderTools). Every call end inside the container and the response goes straight back to the Agent. Azure services (SQL, Blob, Queue, Table) are shared at the bottom.

D. Agent and MCP Client — The Request Path

The user's message is received by an Agent a component in the MCP client application such as VS Code's GitHub Copilot or Claude Desktop that brings its own LLM model and selects the appropriate tool in-process on the client side. The MCP Client then sends a network call over HTTPS to the MCP Server deployed on Azure App Service. The MCP Server receives an already-structured request: a tool name and the extracted arguments. It looks up the corresponding method in ReminderTools and runs it in-process. The server never sees plain language and never makes a routing decision itself.

The MCP Server is completely stateless and model-agnostic. GitHub Copilot, Claude Desktop, and Cursor each bring their own LLM model and the server behaves identically for all of them because the tool selection has already happened on the client side before the network call is made.

E. Shared Azure Resources

The MCP Server connects to four Azure services and one external API, all shown inside the Azure boundary at the bottom of Figure 1 ReminderAgent Cloud Architecture (MCP Server). The user message is received by the Agent (client-side LLM) which selects a tool in-process. The MCP Client sends a network call over HTTPS to the MCP Server deployed on Azure App Service. The server looks up the tool by name and runs it in-process via MCP Tools (ReminderTools). Every call end inside the container and the response goes straight back to the Agent. Azure services (SQL, Blob, Queue, Table) are shared at the bottom.. The four Azure services are Azure SQL Database, Azure Blob Storage, Azure Queue Storage, and Azure Table Storage. OpenAI is a fifth dependency used for embedding generation, hosted externally.

Azure SQL Database hosts two tables. The dbo.Assets table stores the full reminder record: Name, Category, Tags, City, Country, Region, EventDate, TimelineState, UserExperience, UserInput, CreatedAt, and the VECTOR (1536) embedding column as noted in Figure 1 ReminderAgent Cloud Architecture (MCP Server). The user message is received by the Agent (client-side LLM) which selects a tool in-process. The MCP Client sends a network call over HTTPS to the MCP Server deployed on Azure App Service. The server looks up the tool by name and runs it in-process via MCP Tools (ReminderTools). Every call end inside the container and the response goes straight back to the Agent. Azure services (SQL, Blob, Queue, Table) are shared at the bottom. Photos Table stores path only + embeddings (VECTOR). The dbo.Photos table stores one row per uploaded photo, containing AssetId (foreign key), BlobName (GUID plus extension, NOT NULL), BlobUrl (permanent storage URL, NOT NULL), AccessKey (SAS token), FileName, ContentType, and UploadedAt.

Azure Blob Storage [10] holds photo files and bulk-upload files as labelled in Figure 1 ReminderAgent Cloud Architecture (MCP Server). The user message is received by the Agent (client-side LLM) which selects a tool in-process. The MCP Client sends a network call over HTTPS to the MCP Server deployed on Azure App Service. The server looks up the tool by name and runs it in-process via MCP Tools (ReminderTools). Every call end inside the container and the response goes straight back to the Agent. Azure services (SQL, Blob, Queue, Table) are shared at the bottom.: photo files + bulk-upload files. The container is private; photos are served only via the SAS token stored as AccessKey in dbo.Photos. Azure Queue Storage [11] receives bulk-upload job messages processed by the async background worker labelled in Figure 1 ReminderAgent Cloud Architecture (MCP Server). The user message is received by the Agent (client-side LLM) which selects a tool in-process. The MCP Client sends a network call over HTTPS to the MCP Server deployed on Azure App Service. The server looks up the tool by name and runs it in-process via MCP Tools (ReminderTools). Every call end inside the container and the response goes straight back to the Agent. Azure services (SQL, Blob, Queue, Table) are shared at the bottom.: bulk-upload jobs (async worker). Messages failing more than three times are moved to a poison queue. Azure Table Storage [12] hosts BulkUploadStatus and ActivityLog labelled in Figure 1 ReminderAgent Cloud Architecture (MCP Server). The user message is received by the Agent (client-side

LLM) which selects a tool in-process. The MCP Client sends a network call over HTTPS to the MCP Server deployed on Azure App Service. The server looks up the tool by name and runs it in-process via MCP Tools (ReminderTools). Every call end inside the container and the response goes straight back to the Agent. Azure services (SQL, Blob, Queue, Table) are shared at the bottom.: job status + activity log (status only). ReminderTools calls all four Azure services through interface implementations injected at startup, encapsulated behind the six service interfaces defined in ReminderAgent.Common. Table 1 summarises all five services.

Table 1 summarises all five services, their configuration, and their purpose in the system.

Azure Service	Configuration	Purpose
Azure SQL Database	VECTOR (1536), ON DELETE CASCADE FK	dbo.Assets + dbo.Photos + embeddings
Azure Blob Storage	Private photos/ container	Photo files, SAS token access
Azure Queue Storage	bulk-upload-queue + poison queue	Async processing, 3-retry limit
Azure Table Storage	BulkUploadStatus + ActivityLog	Job tracking + activity log
OpenAI API	text-embedding-3-small + gpt-4o-mini	Embeddings + NL understanding

Table 1 Azure services and configuration

F. Database Schema

The SQL schema uses two tables with referential integrity. dbo.Assets stores all reminder data including the VECTOR (1536) [13] embedding column. TimelineState (Past, Present, Future) is computed in C# from EventDate at query time. Tags are semicolon-delimited NVARCHAR. dbo.Photos references Assets via ON DELETE CASCADE deleting an asset automatically removes all linked photo rows. dbo.Photos stores both BlobName (NVARCHAR (500) NOT NULL) and BlobUrl (NVARCHAR (2048) NOT NULL). BlobName is used for storage-layer operations such as deletion and download; BlobUrl holds the permanent base URL used when constructing access links for the caller. Both columns are required and neither is nullable.

G. The Nine MCP Tools

ReminderTools exposes nine methods, each decorated with [McpServerTool] and [Description] attributes. The [McpServerTool] attribute registers the method with the MCP SDK [14] so it appears in the tool list returned to any connecting client. The [Description] attribute is the text the client's language model reads at runtime to decide which tool to invoke and which arguments to extract from the user's message. Every tool call is logged to Azure Table Storage ActivityLog via ITableStatusService.LogActivityAsync regardless of success or failure, providing a permanent audit trail of all agent operations.

1. CreateAsset saves a new reminder to Azure SQL dbo.Assets with full metadata name, category, tags, city, country, region, event date, and user experience notes. Persistence happens in two sequential steps. First, CreateAssetAsync executes an INSERT INTO dbo.Assets statement, creating the row with a new GUID as its Id and leaving the Embedding column as NULL. Second, EmbeddingService calls OpenAI text-embedding-3-small to generate a 1,536-dimensional float vector, and SaveEmbeddingAsync executes UPDATE dbo.Assets SET Embedding = @Embedding WHERE Id = @Id to fill in the VECTOR (1536) column on the row that was just created. The TimelineState field (Past, Present, or Future) is computed from EventDate before the INSERT and stored as part of the initial row, not derived at query time.

2. SearchAssets is the unified search entry point that replaced four separate tools from the SE project (SearchAssets, ListAssets, GetReminders, and SearchRemindersSemantic consolidated into one). It first pulls all assets from Azure SQL and applies structured pre-filters category, city, country, and timeline as in-memory filtering. This is cheap relative to embedding computation because it involves no vector arithmetic or OpenAI API calls. If a free-text query is provided and semantic mode is not explicitly requested, keyword ranking is applied first. If keyword search returns zero results, the tool automatically falls back to semantic mode, so the user never receives a silently empty response. When semantic=true is set explicitly, cosine similarity ranking is applied immediately, with an automatic fallback to keyword-only search if the embedding call itself fails for example due to a network error or OpenAI rate-limit.

3. GetAssetDetails retrieves the full record for a named asset from Azure SQL, including all linked photo references from dbo.Photos. Rather than generating a fresh SAS URL on each read, it reuses the AccessKey that persisted at upload time. When AttachPhoto runs, it requests a one-year-expiry SAS token from IPhotoStorageProvider.GetPhotoAccessKeyAsync and stores that token as the AccessKey field in dbo.Photos alongside the permanent BlobUrl. GetAssetDetails reads both fields and concatenates them to construct the access link returned to the caller. This means the access link is only as fresh as the upload date a photo attached more than one year ago would return an expired link, since no SAS token refresh occurs on read.

4. AttachPhoto validates the local file path and file extension against an allow-list (jpg, jpeg, png, gif, webp) before uploading to Azure Blob Storage. The blob is named as a GUID combined with the original file extension. The BlobName, the permanent BlobUrl, and a freshly generated SAS token (stored as AccessKey) are all written together to a single row in dbo.Photos. In the current implementation, BlobName is used only for storage-layer operations such as deletion and direct download. Links served to the user for example by GetAssetDetails are constructed from the stored BlobUrl and AccessKey fields rather than reconstructed from BlobName.

5. DeleteImage accepts either an asset name or a photo GUID as input using case-insensitive matching. If the asset has a single photo, it is deleted immediately from both Blob Storage and dbo.Photos. If the asset has multiple photos the

tool asks the user to specify a filename to avoid accidental deletion. If a photo GUID is provided directly, only that specific photo is removed.

6. DeleteAsset explicitly deletes each linked blob from Azure Blob Storage and removes each corresponding dbo.Photos row first, then deletes the asset row from dbo.Assets. The ON DELETE CASCADE constraint on dbo.Photos exists as a safety net for any photo rows that may have been missed, but it is not the primary mechanism removing photo rows on this path the tool handles that explicitly before the parent row is deleted.

7. BulkUploadAssets uploads a CSV, JSON, or ZIP file to Azure Blob Storage and enqueues a BulkJobMessage containing the JobId and BlobName to the bulk-upload-queue. The tool returns immediately with the Job ID the user does not wait for the actual row processing, which happens asynchronously in the background worker.

8. GetBulkUploadStatus reads the job summary row from Azure Table Storage BulkUploadStatus using the Job ID, returning the total row count, succeeded count, failed count, and completion timestamp. If the job is still in progress, no summary row exists yet.

9. GetRecentActivity reads the most recent entries from Azure Table Storage ActivityLog and returns them in newest-first order. The reverse-tick RowKey pattern computed as DateTime.MaxValue.Ticks minus the current UTC ticks ensures newest entries always sort first without requiring additional sorting or filtering on the client side.

H. Semantic Search

On CreateAsset, EmbeddingService concatenates the asset Name, Category, UserExperience, UserInput, and Tags into a structured text string and calls OpenAI text-embedding-3-small to produce a float [1536] vector, which is stored in the Assets.Embedding column of type VECTOR (1536) in Azure SQL. On semantic search, a query embedding is generated from the user's search text using the same model, and SimilarityService computes the cosine similarity between the query vector and every stored asset vector:

$$sim(A, B) = (A \cdot B) / (\|A\| \cdot \|B\| + \varepsilon)$$

where ε = 1e-8 prevents division by zero for zero-magnitude vectors. Assets scoring above 0.5 are returned ranked by descending score. SearchAssets automatically falls back to semantic mode when keyword search returns zero results, ensuring the user never receives a silently empty response.

The similarity scoring is performed inside the .NET application in C# rather than inside Azure SQL. Under a production vector search design, similarity would be computed directly in a SQL query using a built-in distance function. However, the Microsoft.Data.SqlClient 7.x preview driver required to write and read VECTOR (1536) columns does not yet expose vector distance functions for use inside SQL queries, as this capability is still being developed by Microsoft. As a result, GetAllEmbeddingsAsync loads all stored embeddings from dbo.Assets into application memory, and SimilarityService.CosineSimilarity scores each one against the query vector in C# code. For the current development dataset this is acceptable. At production scale this would be replaced with Azure AI Search, which maintains a dedicated vector index and computes server-side approximate nearest-neighbour similarity without transferring embedding data to the application.

I. AI Usage and Development Workflow

Two generative AI tools were employed throughout the development lifecycle: Claude Sonnet 4.6 (Anthropic) via conversational prompting for architecture design, implementation generation, and technical documentation; and GitHub Copilot Chat with GPT-4o backend integrated into Visual Studio for inline method completion and real-time error diagnosis. Two additional OpenAI models operate at runtime and are not development tools: text-embedding-3-small generates the 1,536-dimensional semantic vectors during asset persistence and query execution, and GPT-4o-mini handles natural language intent routing in the console REPL entry point. Table 2 documents all four models with their exact versions as used during development and deployment.

AI Tool	Version	Primary Role
Claude (Anthropic)	claude-sonnet-4-6	Architecture, full code generation, docs
GitHub Copilot	GPT-4o (VS integration)	Inline completion, error explanation
OpenAI text-embedding	text-embedding-3-small	Runtime 1,536-dim vector generation
OpenAI Chat	gpt-4o-mini	Runtime NL understanding and tool routing

Table 2 AI tools and versions

I.1 AI-Generated Components (~70% of total codebase)

The following components were generated in their entirety by Claude Sonnet 4.6 from interface contracts, entity definitions, and descriptive requirements provided by the team. In each case the team supplied the interface definition and schema as the specification; the AI produced the implementation body.

SqlDataAccessLayer.cs was generated in full of the IDataAccessLayer interface and schema.sql. All methods including CreateAssetAsync, GetAllAssetsAsync, SearchAssetsAsync, SaveEmbeddingAsync, GetAllEmbeddingsAsync, InsertPhotoAsync, DeletePhotoAsync, and DeleteAssetAsync were generated with correct ADO.NET command construction, parameterised queries, and connection lifetime management.

AzureBlobPhotoStorageProvider.cs was generated from IPhotoStorageProvider, producing correct BlobContainerClient usage for upload with Content-Type header propagation, stream-based download, idempotent delete via DeleteIfExistsAsync, and SAS URL generation with configurable expiry. AzureQueueService.cs was generated from IQueueService, producing correct QueueClient instantiation with CreateIfNotExists, SendMessageAsync, ReceiveMessagesAsync with visibility timeout, and DeleteMessageAsync.

AzureTableStatusService.cs was generated from ITableStatusService. The reverse-tick RowKey pattern for ActivityLog computed as DateTime.MaxValue.Ticks minus DateTime.UtcNow.Ticks to achieve newest-first ordering without secondary sorting was introduced by the AI unprompted, demonstrating domain knowledge beyond direct instruction.

ReminderTools.cs was generated with all nine [McpServerTool]-decorated methods, correct [Description] attribute text for MCP client tool discovery, and registration via AIFunctionFactory.Create for the console REPL path. Program.cs dependency injection wiring for both startup projects was generated including correct service lifetime selection, configuration binding, and the background worker Task.Run pattern with cancellation token propagation. schema.sql was generated as a complete DDL script including VECTOR (1536) column declaration, ON DELETE CASCADE foreign key constraint, and non-clustered index definitions on Category, Country, and AssetId. Integration test class scaffolding was generated including ClassInitialize, TestCleanup, ClassCleanup, and the unique run-identifier pattern for test data isolation.

I.2 Human-Authored Components (~30% of total codebase)

The following components were authored by the team without AI generation. In each case the decision to write manually reflected an architectural judgment that the component required design choices the team could not fully specify as a prompt, or that the component defined the contract against which all generated code would be measured.

All six service interface files IDataAccessLayer, IPhotoStorageProvider, IQueueService, ITableStatusService, IEmbeddingService, and ISimilarityService were authored by the team. Interface design requires decisions about method granularity, parameter contracts, return type semantics, and the boundary between what a caller must know and what an implementation may vary. These decisions propagate to every generated implementation and every test assertion; delegating them to a generative model would have produced interfaces shaped by the model's assumptions rather than the project's requirements.

The Asset.cs entity class was authored by the team because field selection, nullability decisions, and the choice to compute TimelineState in C# rather than store it as a SQL computed column required explicit architectural judgment. BulkFileParser.cs was authored manually because ZIP archive extraction, multi-format detection, and row-level error isolation logic required implementation decisions specific to the file formats supported. InitHelpers.cs was authored manually to implement the three-layer configuration strategy appsettings.json, appsettings.dev.json, environment variables with the correct override precedence for both local development and Azure App Service deployment. MyConfig.cs was authored manually to centralise all Azure resource names, eliminating the hardcoded string literals that appeared in AI-generated provider implementations. All integration test Assert conditions and expected-value strings were authored manually after reviewing the actual tool return contracts, because AI-generated assertions repeatedly used incorrect expected strings that did not match the real tool output format.

I.3 AI Output Requiring Correction

Twelve categories of issues were identified in AI-generated code, architecture, and configuration across the full development lifecycle. They fall into four thematic groups: type handling and missing runtime behavior, architectural mismatches, logic defects in tool behavior, and unprompted additions with incorrect test assertions.

Type Handling and Missing Runtime Behaviour

The most impactful issue was in SqlDataAccessLayer.cs, where the generated code used Buffer.BlockCopy byte-array serialization for the Embedding column. This approach is correct for VARBINARY(MAX) but incompatible with Azure SQL's native VECTOR (1536) type, which uses a different internal representation. The generated code was structurally correct in method signature and control flow but failed at the type-specific level because the AI's training data predates the VECTOR preview type. The fix required removing the Buffer.BlockCopy conversion entirely and using the correct SDK parameter path for the native VECTOR column.

A related failure appeared in event date handling. When the user typed a message such as "remind me to buy groceries tomorrow", the EventDate was correctly extracted from the sentence and appeared in the console response but was never written to the SQL row. The AI separated argument extraction from persistence without connecting them the resolved date existed in memory but was discarded before the INSERT statement executed. Location inference exhibited the same disconnect typing "Luxembourg city" did not automatically populate the Country and Region fields despite the AI implying this would work through GPT-4o-mini geographic inference.

The bulk upload worker exhibited a third instance of this pattern. The AI-generated architecture bypassed CreateAsset entirely when processing bulk rows, silently skipping event date parsing and TimelineState computation for every bulk-imported row. The fix required adding explicit date parsing inside the worker loop, running inside the MCP Server process, so that bulk-imported rows receive the same classification as rows created directly through the MCP Server.

Architectural Mismatches

ReminderAgent.McpServer was initially generated as a plain Web API with an endpoint named /mcp not a real Model Context Protocol server. The endpoint accepted HTTP requests but did not implement the MCP protocol, meaning no MCP-compatible client could connect and discover tools. The team identified this before shipping and rebuilt the project using the real ModelContextProtocol SDK over Streamable HTTP.

A second architectural gap was that the background queue worker was initially generated only inside the console application's Program.cs rather than the MCP Server's. Since bulk upload jobs are enqueued exclusively through the MCP Server in production, a job enqueued there would sit in the queue with no worker to pick it up. The fix required moving

the worker loop into ReminderAgent.McpServer so the single production process both accepts MCP tool calls and processes bulk upload jobs.

Logic Defects in Tool Behaviour

The DeleteImage tool was generated with a delete-by-asset-id query rather than a delete-by-photo-id query. When an asset had two photos attached and the user asked to delete one specific photo, both were removed. The correct behaviour is to match on the individual photo reference by filename or GUID and leave all other photos intact. The fix required rewriting the deletion logic to target a single row in dbo.Photos rather than all rows for the asset.

GetAssetDetails reported reminders as having photos attached even when no photos existed in Azure Blob Storage. The generated code used an in-memory flag rather than querying the Azure Photos table as the source of truth. The fix was to query dbo.Photos directly within GetAssetDetails. When the team raised this, the AI initially proposed creating a new dedicated tool; the team correctly identified that the fix belonged inside the existing GetAssetDetails method. Additionally, eleven tool methods were initially registered in ReminderTools, several of which were near duplicates with overlapping responsibilities. Human review of the full tool list was required to consolidate them to nine tools with clearly distinct purposes.

Unprompted Additions and Incorrect Test Assertions

API key authentication using an X-Api-Key header was added to the MCP endpoint without being requested. The addition broke all MCP client connections immediately and was reverted entirely. The logging infrastructure required multiple correction rounds: InitHelpers.cs was first generated as an empty stub, then populated with AddSimpleConsole and IncludeScopes that did not match the reference implementation provided, then corrected to use AddSimpleConsole, and finally simplified to remove console logging entirely because the debug output window already provided sufficient visibility during development.

Several integration test assertions were generated with incorrect expected strings and wrong protocol assumptions. Testcases TOOL-09, TOOL-16, and TOOL-17 checked for "couldn't find a saved asset" but the actual return value under timeout conditions was a SQL connection error message. MCP-13 and MCP-14 treated normal successful tool responses a bulk upload job confirmation and a not-found acknowledgement as protocol errors by asserting result.IsError on responses that are valid tool outputs, not failure conditions. All assertions required manual correction against the actual tool return contract.

IV. RESULTS

A. System Deliverables

All planned system components were successfully implemented and verified against Azure infrastructure. The Azure SQL schema was deployed using schema.sql, creating dbo.Assets with a native VECTOR (1536) embedding column and dbo.Photos with an ON DELETE CASCADE foreign key to dbo.Assets. Both tables carry non-clustered indexes on the most frequently queried columns Category, Country, and AssetId to optimise common filter patterns without full table scans.

All nine MCP tools are implemented in ReminderTools, decorated with [McpServerTool] and [Description] attributes, and registered as AIFunctions via AIFunctionFactory. The MCP Server endpoint at /mcp is live and responds correctly to the JSON-RPC initialize handshake, returning all nine tool names with their descriptions to any connecting client. The internal development harness (console REPL) was used throughout implementation to validate ReminderTools business logic against real Azure services before that logic was exercised through the MCP wire protocol. Because both paths call the identical ReminderTools methods, this internal validation gives confidence that the single production entry point the MCP Server behaves correctly, without requiring the console itself to be part of the deployed system.

Azure Blob Storage is fully operational: photos are uploaded to the private photos/ container, downloaded with byte-level fidelity, deleted idempotently, and SAS URLs are generated with correct expiry encoding and Read-only permission. Azure Queue Storage is operational with send, receive, and delete working correctly, and the poison queue correctly receives messages that exceed three processing attempts. Azure Table Storage ActivityLog correctly records every tool invocation in newest-first order using the reverse-tick RowKey pattern, and BulkUploadStatus correctly stores both per-row status and job summary records.

ReminderAgent.Cloud runs both the console chat loop and the background queue worker simultaneously in the same process without either task blocking the other. The background worker picks up bulk upload jobs within five seconds of enqueuing, processes each row as an Asset with a persisted embedding, and writes a complete job summary to Table Storage on completion. The entire bulk upload pipeline from user file upload through to queryable job status was verified end-to-end using a CSV file against Azure services.

B. Test Suite

The test suite is structured as four MSTest classes, each targeting a distinct architectural layer of the system. All 51 test methods execute against real provisioned Azure services Azure SQL Database, Azure Blob Storage, Azure Queue Storage, and Azure Table Storage with no infrastructure stubbing or in-memory substitution except where the code path under test provably does not invoke a given service. The suite follows the principle of minimal substitution: test doubles are introduced only when their absence would incur external API cost for calls the system under test does not make, not to simplify assertion logic or avoid environment dependencies.

Test isolation is enforced through two mechanisms. First, every test class is annotated with [DoNotParallelize], which instructs the MSTest runner to execute classes sequentially rather than concurrently. This was found necessary after parallel execution exhausted the Azure SQL connection pool, causing connection acquisition timeouts that produced false negatives indistinguishable from genuine defects. Second, every test class generates a unique six-character

hexadecimal run identifier at ClassInitialize time using Guid.NewGuid(). ToString("N") [.6]. This identifier is embedded in all asset names and file paths created during the run, preventing row-level data collisions between test executions on the shared development database. All created rows, blobs, and queue messages are removed deterministically in [TestCleanup] and [ClassCleanup], leaving all Azure resources in their pre-test state.

C. Test Class Descriptions

RemindertoolTests — Functional Verification Layer (18 Tests)

RemindertoolTests instantiates ReminderTools against a fully wired dependency injection container containing real implementations of all six service interfaces. No service is substituted. The eighteen test methods exercise the complete CreateAsset execution path, verifying that the asset row persisted to dbo.Assets with correct field values and that the 1,536-dimensional embedding vector is generated via OpenAI text-embedding-3-small and stored in the VECTOR (1536) column. Timeline classification is verified for both past and future event dates. Input validation is verified by asserting that an empty name argument produces an appropriate error response without throwing an unhandled exception. SearchAssets is exercised with category filters, timeline filters, and semantic similarity mode. GetAssetDetails is verified for both a known asset name and an unknown name. Six additional test methods exercise the TimelineHelpers and DuplicateAssetHelpers utility classes in isolation, covering date format parsing, timeline state computation, fuzzy name matching, exact duplicate detection, and field merge semantics when a duplicate asset is updated with partially populated data.

ReminderTools_PhotoOperations_Tests — Photo Tool Verification Layer (8 Tests)

ReminderTools_PhotoOperations_Tests verifies the AttachPhoto and DeleteImage tool methods against real Azure SQL Database and Azure Blob Storage. IEmbeddingService and ISimilarityService are substituted with Moq test doubles because neither AttachPhoto nor DeleteImage invokes those interfaces at any point in their execution paths. Substituting them eliminates an unnecessary dependency on the OpenAI API for tests that do not exercise embedding functionality, without compromising the fidelity of the storage layer under test. The eight test methods cover pre-upload file extension validation, rejection of non-existent file paths, asset lookup failure propagation, the successful round-trip of a valid JPEG through UploadPhotoAsync and InsertPhotoAsync into dbo.Photos, SAS URL accessibility verification, and DeleteImage resolution behavior for assets with a single attached photo, multiple attached photos requiring filename disambiguation, and deletion by photo GUID.

AzureBlobPhotoStorageProviderTests — Storage Provider Verification Layer (8 Tests)

AzureBlobPhotoStorageProviderTests exercises AzureBlobPhotoStorageProvider directly against the provisioned Azure Blob Storage photos/ container, bypassing the ReminderTools abstraction layer entirely. No service interface is substituted. The eight test methods verify that UploadPhotoAsync returns a BlobName conforming to the expected GUID-plus-extension format and sets the correct Content-Type metadata header on the uploaded blob. DownloadPhotoAsync is verified for byte-level round-trip fidelity against the uploaded content. DeletePhotoAsync is verified for both successful blob removal and idempotent handling of a non-existent blob name without raising an exception. GetPhotoAccessKeyAsync is verified to return an HTTPS-scheme SAS URL that yields HTTP 200 when fetched with an HttpClient, and HTTP 403 after the configured expiry period elapses.

McpServer_Protocol_Tests — End-to-End Protocol Verification Layer (17 Tests)

McpServer_Protocol_Tests provides the deepest level of integration coverage in the suite. ClassInitialize launches ReminderAgent.McpServer as a real operating system child process, bound to a dynamically allocated loopback port obtained from the OS via TcpListener. A polling loop with a ninety-second budget issues HTTP GET requests to the server health endpoint at half-second intervals until a 200 OK response confirms the ASP.NET Core host, MCP SDK middleware, and Azure service connections are fully initialized. A Streamable HTTP McpClient connects to the /mcp endpoint for the duration of the class. A separate SqlDataAccessLayer instance is initialized for out-of-band database verification asserting that tool calls over the MCP wire produce real SQL row mutations, not merely plausible-looking text responses. The seventeen test methods exercise the full MCP tool surface, from protocol initialization and tool list discovery through concurrent client execution, DeleteAsset cascade behavior, and graceful not-found handling verified by a subsequent ListTools call confirming the server remained in a healthy state after the failed lookup.

D. Test Results Visualised

Figure 2 illustrates the distribution of 51 test methods across the four classes. RemindertoolTests accounts for the largest proportion at 18 tests because it covers not only the nine MCP tool methods but also the TimelineHelpers and DuplicateAssetHelpers utility classes that underpin correct asset classification and idempotent write behaviour. McpServer_Protocol_Tests contributes 17 tests reflecting the breadth of tool surface area exercised at the JSON-RPC wire protocol level. PhotoOperations and BlobStorage classes each contribute 8 focused tests targeting their respective Azure service boundaries.

Fig. 2. Integration Test Distribution Across Four Test Classes

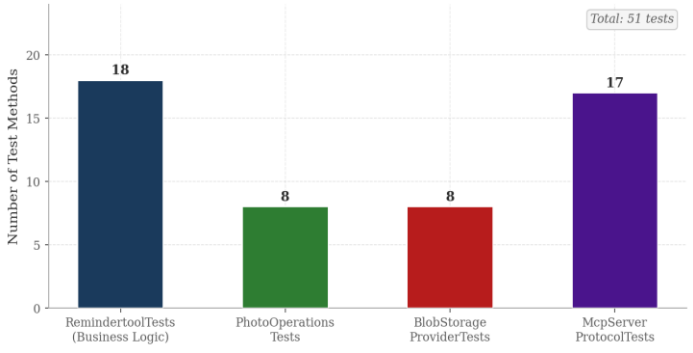


Figure 2. Distribution of 51 integration test methods across four MSTest classes. RemindertoolTests (18) covers tool methods and utility classes. McpServer_Protocol_Tests (17) exercises the full MCP wire protocol.

Figure 3 presents the test substitution strategy across all four classes. Forty-three of the 51 test methods execute against real provisioned Azure services with no substitution at any layer. Eight test methods in ReminderTools_PhotoOperations_Tests substitute IEmbeddingService and ISimilarityService with Moq test doubles because those interfaces are not on the execution path of AttachPhoto or DeleteImage. The 84% real-infrastructure ratio reflects the deliberate strategy of validating actual cloud service behavior rather than simulated responses, accepting the trade-off of network-bound execution time in exchange for higher confidence in production equivalence.

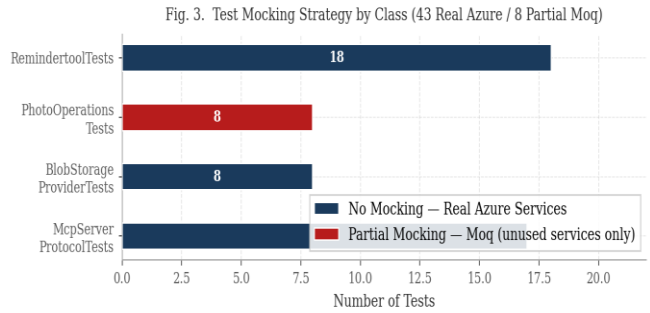


Figure 3. Test substitution strategy. 43 of 51 test methods use no substitution real Azure services at every layer. 8 tests substitute interfaces not on the execution path of the code under test.

Figure 4 presents Azure service and protocol coverage aggregated across all test classes. Azure SQL Database receives the highest coverage at 26 test methods because asset row persistence is verified independently by both the functional layer and the out-of-band DAL in the protocol layer. Azure Blob Storage follows at 25 tests spanning upload, download, deletion, and SAS token generation verification. The MCP wire protocol, Azure Queue Storage, and Azure Table Storage are each exercised by 17 test methods through McpServer_Protocol_Tests. OpenAI Embeddings is invoked by the 18 RemindertoolTests that exercise CreateAsset and the semantic variant of SearchAssets against the API endpoint.

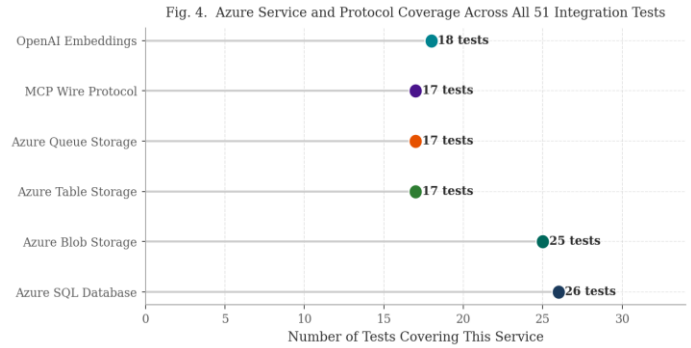


Figure 4. Azure service and protocol coverage across all 51 test methods. Azure SQL Database (26) and Blob Storage (25) receive the highest coverage. MCP Wire Protocol, Queue Storage, and Table Storage each exercised by 17 tests.

E. Deployment Evidence

ReminderAgent.McpServer was containerized using Docker, pushed to Azure Container Registry (mcpserverreminderagent, Germany West Central) under the image tag mcpserverreminderagent.azurecr.io/reminderagent-mcpserver:latest, and deployed to Azure App Service (reminderagent, North Europe, App Service Plan: TechTrio). The following evidence confirms successful end-to-end deployment and correct operation of all system components against real Azure infrastructure.

E.1 Application Health and MCP Endpoint Availability

Figure 5 shows the Azure App Service overview confirming the container is in Running state under the TechTrio App Service Plan in the North Europe region. The deployed application responds correctly to the health check endpoint: an HTTP GET request to https:reminderagent-cta7e8fegycye8gh.northeurope 01.azurewebsites.net returns HTTP 200 with the response body "ReminderAgent MCP Server is running. MCP endpoint: /mcp", confirming that the ASP.NET Core host, MCP SDK middleware, and Azure service connections initialized successfully on container startup. Figure 6 shows this health check response in the browser, and Figure 7 confirms the /mcp endpoint responds correctly to the JSON-RPC initialize handshake, returning all nine tool definitions to the connecting MCP client.

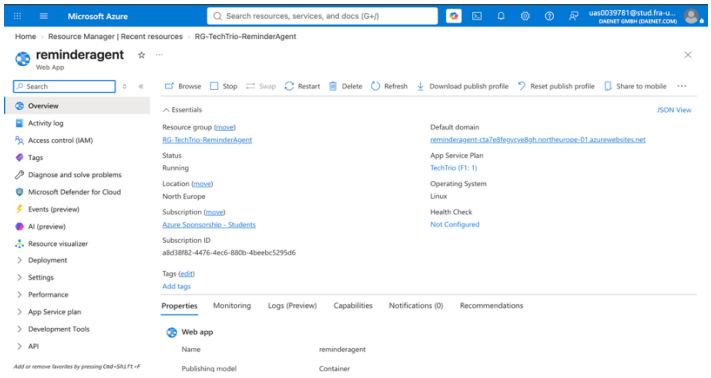


Figure 5. Azure Portal App Service overview showing Status: Running, URL:https://reminderagent-cta7e8fegycye8gh.northeurope-01.azurewebsites.net.,App Service Plan: TechTrio, Region: North Europe

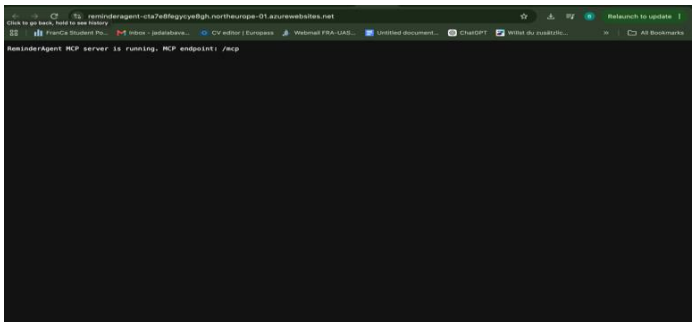


Figure 6. Browser showing health check response: "ReminderAgent MCP Server is running. MCP endpoint: /mcp"

E.2 Azure Container Registry

Figure 7 shows the Azure Container Registry (mcpserverreminderagent) repository view confirming the Docker image was built from ReminderAgent.McpServer and pushed successfully. The repository shows the reminderagent-mcpserver image with the v2 tag and the push timestamp, confirming the full CI pipeline from source code through containerization to registry storage completed without error.

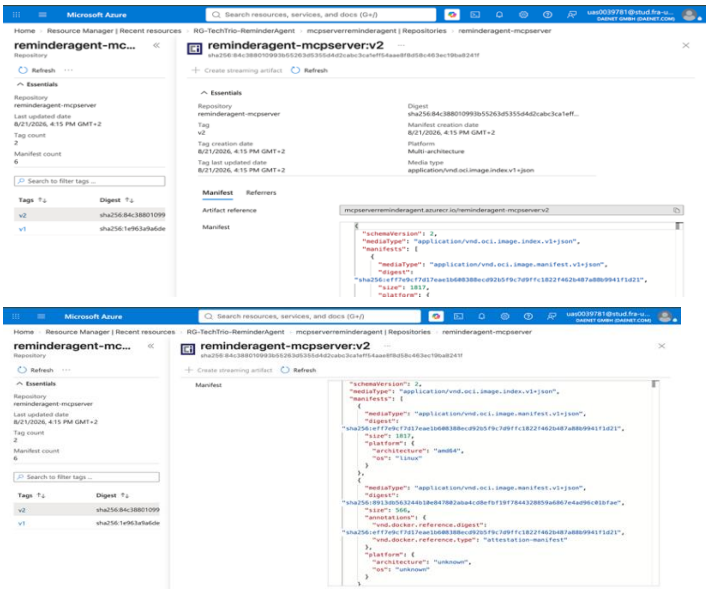


Figure 7. Azure Portal Container Registry showing repository reminderagent-mcpserver with latest tag and push timestamp

E.3 Azure SQL Database — Data and Embedding Persistence

Figure 8 shows the Azure Portal Query Editor result for the following verification query executed against ReminderAgentDb: SELECT Id, Name, Category, City, TimelineState, DATALENGTH(Embedding) AS EmbeddingBytes, CreatedAt FROM dbo.Assets ORDER BY CreatedAt DESC.

The result confirms that asset rows persisted with correct field values and that DATALENGTH(Embedding) returns 6,152 bytes for every row equivalent to 1,536 single-precision floats at 4 bytes each proving that the full OpenAI text-embedding-3-small vector is stored in the native VECTOR (1536) column for every saved reminder.

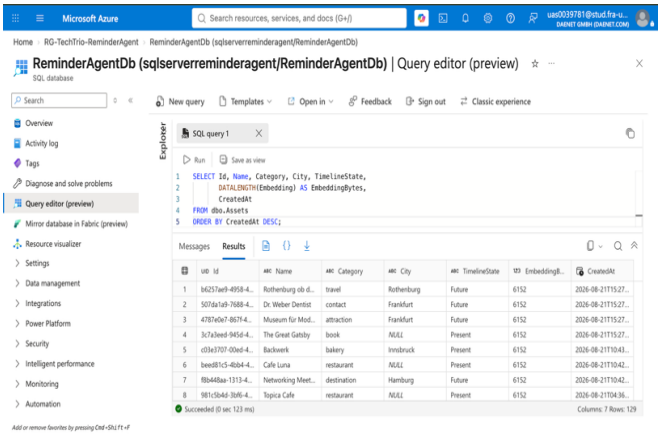


Figure 8. Azure Portal Query Editor showing SELECT result with asset rows and EmbeddingBytes = 6152 per row

E.4 Azure Table Storage — Activity Log

Figure 9 shows the ActivityLog table in Azure Table Storage, accessed via the Azure Portal Storage browser. Every tool invocation across both the local test runs and the deployed MCP Server is recorded as a row with PartitionKey set to the UTC date, RowKey set to the reverse-tick value ensuring newest-first ordering, ToolName identifying which of the nine tools was called, and a Success flag confirming the outcome. The log confirms that CreateAsset, SearchAssets, AttachPhoto, DeleteImage, and GetRecentActivity were all invoked and recorded successfully.

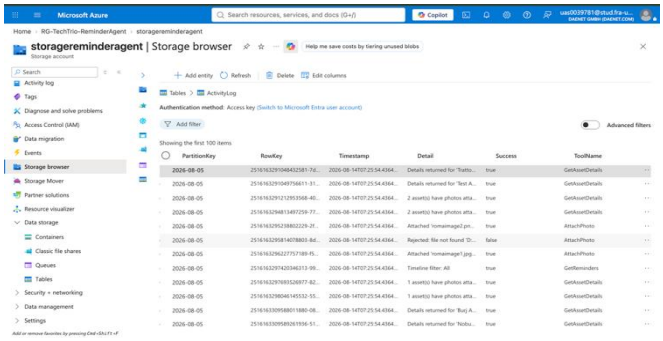


Figure 9. Azure Portal Storage browser showing ActivityLog table with rows for CreateAsset, SearchAssets, and other tool invocations with UTC timestamps and Success = true

E.5 App Service Metrics

Figure 10 shows the Azure App Service metrics dashboard for the reminderagent deployment. The Requests metric confirms HTTP requests were received and processed by the deployed container. The Average Response Time metric shows the latency of MCP tool calls routed through the ASP.NET Core host. The Memory Working Set metric confirms the container operates within the memory allocation of the TechTrio App Service Plan. These metrics collectively confirm the deployed service handled real MCP client traffic after deployment.

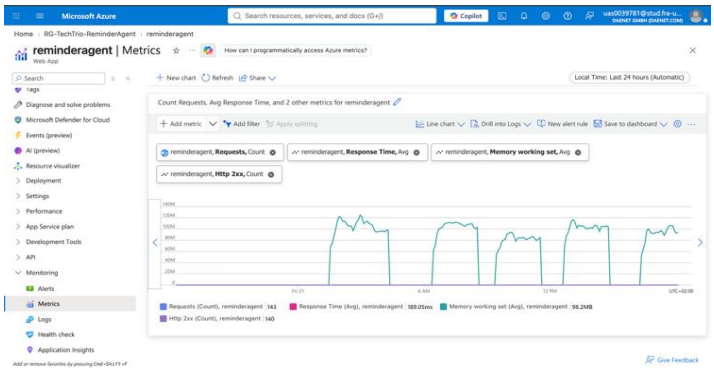


Figure 10. Azure Portal App Service Metrics showing Requests count, Average Response Time, HTTP 2xx, and Memory Working Set graphs

E.6 MCP Client Connected to Deployed Endpoint

Figure 11 shows an MCP-compatible client Claude Desktop or GitHub Copilot Chat connected to the live deployed endpoint at https://reminderagent-cta7e8fegycye8gh.northeurope-01.azurewebsites.net. The

tool list returned by the server shows all nine ReminderTools methods available for invocation: CreateAsset, SearchAssets, GetAssetDetails, AttachPhoto, DeleteImage, DeleteAsset, BulkUploadAssets, GetBulkUploadStatus, and GetRecentActivity. This confirms that the MCP SDK tool registration, [McpServerTool] attribute decoration, and Streamable HTTP transport are all functioning correctly in the production container environment.

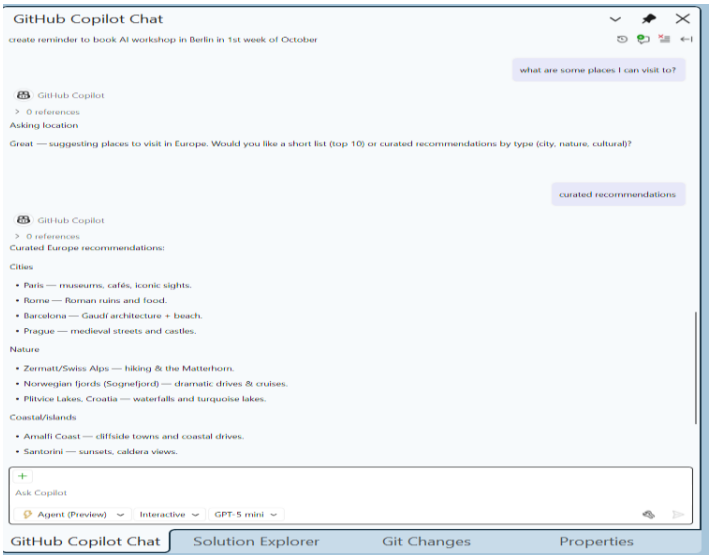


Figure 11. MCP client (Claude Desktop or GitHub Copilot) showing all 9 ReminderAgent tools listed after connecting to https://reminderagent-cta7e8fegycye8gh.northeurope-01.azurewebsites.net

V. DISCUSSION

A. *What Worked Well*

• Single shared ReminderTools library: MCP Server exercises identical business logic and Azure service calls as the local development entry point testing one validates the other.

• Docker containerisation and App Service deployment: ReminderAgent.McpServer containerised and deployed successfully to Azure App Service (reminderagent, North Europe) via Azure Container Registry (mcpserverreminderagent); ReminderAgent.Cloud is intentionally excluded via .dockerignore since only the McpServer image is built and shipped for the production container, keeping the image smaller and avoiding console-only dependencies in the deployed artifact.

• MCP tools auto-discoverable from [Description] attributes by all tested clients, GitHub Copilot Chat [18] and Claude Desktop connected to the deployed HTTPS endpoint without any client-side configuration.

• BulkUploadAssets and AttachPhoto verified successfully end-to-end through Claude Desktop connected to the live deployed endpoint: a CSV file was uploaded and processed by the background worker, and a photo was attached to an existing asset and confirmed retrievable via the returned SAS link.

• DI architecture: identical ServiceCollection in ClassInitialize as in Program.cs eliminated all package resolution and service lifetime issues in the test suite.

• Azure Blob Storage: all four operations (upload, download, delete, SAS generation) correct on first implementation.

• Reverse-tick RowKey pattern in ActivityLog: eliminates query-time sorting; newest entries always first without additional filtering.

• Real child-process MCP server tests: genuine JSON-RPC transport, tool discovery, and concurrent-client coverage that in-process mocking cannot reach.

• SearchAssets automatic semantic fallback: zero keyword results trigger semantic mode transparently, preventing silent empty responses.

• Claude generated correct SqlDataAccessLayer implementations on the first attempt from clear interface contracts and schema.sql. The reverse-tick RowKey for ActivityLog was suggested unprompted. The MCP wire protocol test design launching the real server as an OS child process was Claude's [17] proposal and provides coverage that in-process mocking cannot reach.

B. *Limitations and known gaps*

• VECTOR (1536) column read returns null via Microsoft.Data.SqlClient 7.x preview a confirmed SDK gap; embeddings are written correctly but cannot be read back through the current ADO.NET driver; monitoring release notes for stable support.

• Orphaned blobs after test cleanup: TestCleanup removes SQL rows via ON DELETE CASCADE but cannot delete the underlying Blob Storage files because AssetPhotoRef does not expose BlobName to the cleanup method.

• In-memory cosine similarity does not scale beyond development dataset sizes SimilarityService loads all embeddings per query; production scale requires Azure AI Search vector indexes.

• MCP Server endpoint has no authentication layer the deployed App Service endpoint is publicly accessible; OAuth 2.0 or API key authentication is required before the URL is shared externally.

• AttachPhoto requires a photo path reachable from wherever the tool executes: because the MCP Server is the sole production entry point, the path must be reachable by the Azure App Service container, not the end user's own machine a purely local file path on the user's computer is not accessible to the deployed container.

• AI tools underperformed in three areas: package version awareness (training data predates preview releases causing NU1605 conflicts in every generated .csproj), assembly boundary constraints (AsIEmbeddingGenerator() assumed accessible without correct package reference), and Azure SQL VECTOR type specifics (native VECTOR(1536) conflated with VARBINARY(MAX), generating structurally correct but type-wrong Buffer.BlockCopy serialization code).

C. *Test Design Rationale*

Mocking IEmbeddingService in photo tests avoids OpenAI cost without reducing coverage neither photo tool invokes it. Real child-process McpServer tests ensure ASP.NET Core middleware and Streamable HTTP are exercised as in

production. [DoNotParallelize] was necessary parallel execution exhausted the Azure SQL connection pool.

D. Lessons Learneed

- Interface-first design enables accurate AI generation; clear contracts give models unambiguous targets
- Test immediately after AI generation, type, package, and boundary issues surface at compilation
- AI version awareness lags 6-12 months, verify all generated package versions against NuGet manually
- [DoNotParallelize] is not optional for Azure integration test suites
- Store BlobName not full URL resilient to storage account changes; SAS generated on demand

VI. CONCLUSION

This paper presented the cloud migration of ReminderAgent from a single-machine console application with flat-file storage to a fully deployed cloud-native service on Microsoft Azure. The original SE-project console application is retained only as an internal development and testing tool and is excluded from the production build; the system's single production entry point is a new Model Context Protocol Server that exposes nine tools over a network-accessible endpoint. All stated goals were achieved: Azure SQL Database, Blob Storage, Queue Storage, and Table Storage are integrated and operational; semantic search using 1,536-dimensional OpenAI embeddings is functioning correctly; and the nine MCP tools are accessible from remote AI clients over a live HTTPS endpoint.

ReminderAgent.McpServer has been containerised, pushed to Azure Container Registry (mcpserverreminderagent), and deployed to Azure App Service (reminderagent, North Europe, App Service Plan: TechTrio), with MCP clients including GitHub Copilot Chat and Claude Desktop successfully invoking tools including CreateAsset, SearchAssets, AttachPhoto, and BulkUploadAssets over the live endpoint. The result is a system with exactly one production-facing interface, adding genuine cloud-native accessibility, persistence, and asynchronous processing, without requiring any AI client to clone or run the project locally.

REFERENCES

[1] K. Thabjul, B. Jadala, A.R.Sawant, "ML25/26-11 Implement Reminder Agent," Frankfurt University of Applied Sciences, Apr. 2026 [Online]. Available: https://github.com/UniversityOfAppliedSciencesFrankfurt/se-cloud-2025-2026/tree/TechTrio-ReminderAgent/Source/ML25-26-11-ImplementReminderAgent .

[2] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459-9474. [Online]. Available: https://arxiv.org/abs/2005.11401

[3] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," in Proc. Int. Conf. on Learning Representations (ICLR), 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[4] Microsoft Corporation, "Microsoft Agent Framework — IChatClient, AIFunctionFactory," .NET Blog, 2024. [Online]. Available: https://devblogs.microsoft.com/dotnet/microsoft-extensions-ai-preview

[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proc. NAACL-HLT, 2019, pp. 4171-4186. [Online]. Available: https://arxiv.org/abs/1810.04805

[6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proc. EMNLP-IJCNLP, 2019, pp. 3982-3992. [Online]. Available: https://arxiv.org/abs/1908.10084

[7] OpenAI, "New Embedding Models and API Updates text-embedding-3-small," 2024. [Online]. Available: https://platform.openai.com/docs/guides/embeddings

[8] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, "Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity," in Proc. NAACL-HLT, 2024. [Online]. Available: https://arxiv.org/abs/2403.14403

[9] Model Context Protocol, "MCP Specification v2.1," Anthropic, 2024. [Online]. Available: https://github.com/modelcontextprotocol/specification

[10] Microsoft Corporation, "Azure Blob Storage Documentation," 2024. [Online]. Available: https://docs.microsoft.com/azure/storage/blobs

[11] Microsoft Corporation, "Azure Queue Storage Documentation," 2024. [Online]. Available: https://docs.microsoft.com/azure/storage/queues

[12] Microsoft Corporation, "Azure Table Storage Documentation," 2024. [Online]. Available: https://docs.microsoft.com/azure/storage/tables

[13] Microsoft Corporation, "Azure SQL Database — VECTOR Data Type," Microsoft Learn, 2024. [Online]. Available: https://learn.microsoft.com/azure/azure-sql/database/ai-ready-overview

[14] ModelContextProtocol NuGet Package v2.1.0, 2024. [Online]. Available: https://www.nuget.org/packages/ModelContextProtocol

[15] D. Dobric, "ML25/26-11 Cloud Computing Module Assignment Specification," Frankfurt University of Applied Sciences, 2025.

[16] OpenAI, "GPT-4o-mini Model," 2024. [Online]. Available: https://openai.com/research/gpt-4o-mini

[17] Anthropic, "Claude Sonnet 4.6," 2024. [Online]. Available: https://www.anthropic.com/claude

[18] GitHub, "GitHub Copilot Chat Documentation," 2024. [Online]. Available: https://docs.github.com/copilot/github-copilot-chat

[19] C. Khoo, B. Luyt, C. Ee, J. Osman, H.-H. Lim, and S. Yong, "How Users Organize Electronic Files on Their Workstations in the Office Environment: A Preliminary Study of Personal Information Organization Behaviour," Information Research, vol. 12, no. 2, paper 293, 2007. [Online]. Available: http://informationr.net/ir/12-2/paper293.ht